Rust Intro

CS-3180

New Language

- Many of the remaining concepts we need to cover
- Need at least one more language
- Rust!
- Let's talk motivations first



Many examples lifted from https://doc.rust-lang.org/book/

White House

- Over 50% of CVEs are memory-safety bugs
- National security nightmare
- Feb. 2024 Press Release



BACK TO THE BUILDING BLOCKS:

A PATH TOWARD SECURE AND MEASURABLE SOFTWARE

FEBRUARY 2024



Errors

- Most low level languages (C/++)
- Allow control of memory
- Efficiency and control
- HUGE source of vulnerabilities
- As we learn Rust, memory will play a much more active roll

Hello World

- Let's jump right in
- Declare main with `fn`
- Use `println!` for debugging
- How do we run it?

```
1 fn main() {
2     println!("Hello, world!");
3 }
```

Compilation

- Rust is a *compiled* language
- *rustc* is the compiler
- But you'll probably never use this
- You'll often use a build system instead
- Check out *https://rust-lang.org/tools/install/* for installation
- *rust-analyzer* for VSCode

Build Systems

- *Build systems* are software that manage the execution of your software
- We have stayed away from them so far
- Simplify run commands
- Runtime arguments
- Executable compilation and distribution
- Dependency management

Build Systems

- Languages are mostly de-coupled from their build system
- (e.g. Javascript is not NPM
- Languages often have multiple available
- (e.g. Java has Maven and Gradle)
- The Rust ecosystem is tightly built around the *Cargo* build system

Cargo

- `cargo` command to use cargo
- To create a new project, use `cargo new [project_name]`
- Or initialize an empty directory as a project with `cargo init`
- I tend to also pass the `--vcs=none` flag
- Likely already made the repository

Structure

- You'll be greated with a structure like this
- *Cargo.toml* is where build system properties are managed
- *src/* is where you'll put your code
- `cargo run` to run the project

```
NewProject
├── Cargo.toml
└── src
    └── main.rs
```

Variables and Typing

- Strong and statically typed language
- Will not convert for you
- Checked at compile time
- Rust offloads as much work to the compiler as possible
- `let` keyword to create variables

```
let name: type = value
```

Explicitness

- Rust will *always* require you to be explicit
- Many types of integers
- `i32` + `usize` will take you most of the way

Length	Signed	Unsigned
8-bit	`i8`	`u8`
16-bit	`i16`	`u16`
32-bit	`i32`	`u32`
64-bit	`i64`	`u64`
128-bit	`i128`	`u128`
Architecture-dependent	`isize`	`usize`

Mutability

- Take a look at this code snippet

```
1 fn main() {
2
3     let foo: i32 = 35;
4
5     foo = 38;
6
7     println!("Hello, world!");
8
9 }
```

Mutability

- Take a look at this code snippet
- Gives us a compile-time error

```
1 fn main() {
2
3     let foo: i32 = 35;
4
5     foo = 38;
6
7     println!("Hello, world!");
8
9 }
```

```
➜  Test git:(main) ✗ cargo run
   Compiling Test v0.1.0 (/home/rhatfield/forge/CS-3180/slides
error[E0384]: cannot assign twice to immutable variable `foo`
 --> src/main.rs:5:5
  |
3 |     let foo: i32 = 35;
  |         --- first assignment to `foo`
4 |
5 |     foo = 38;
  |     ^^^^^^^^ cannot assign twice to immutable variable
  |
help: consider making this binding mutable
  |
3 |     let mut foo: i32 = 35;
  |         +++
```

Compiler

- All data in Rust is *immutable* by default
- Opt in to mutability with the `mut` keyword
- Compiler is *extremely* helpful in this regard
- Excellent type inference, can omit more than not

```
1 fn main() {
2
3     let mut foo: i32 = 35;
4
5     foo = 38;
6
7     println!("Hello, world!");
8
9 }
```

Macros

- Rust has an extensive procedural macro system
- We will not go into depth with this
- But macros denoted with a `!`
- `println!` is actually a macro
- This does support AST modification macros
- Call them like functions

Conditions

- `if` statements do not need parentheses
- Work like most procedural conditionals do

```
fn main() {
    let number = 6;

    if number % 4 == 0 {
        println!("number is divisible by 4");
    } else if number % 2 == 0 {
        println!("number is divisible by 2");
    } else {
        println!("number is not divisible by 4 or 2");
    }
}
```

Interpolation

- Strings are a little tricky (more on this later)
- Can interpolate in `println!` just like in Python
- Use `{variable}` or `{}`, then pass as arguments

```
1 fn main() {
2     let x = 5;
3     let y = 10;
4
5     println!("x = {x} and y + 2 = {}", y + 2);
6 }
```

Functions

- Use the `->` to denote a return type
- Use the `return` keyword

```
1  fn main() {
2      let a: i32 = 35;
3      let b: i32 = 28;
4
5      let result = add(a, b);
6      println!("Result was {}", result);
7
8  }
9
10 fn add(a: i32, b: i32) -> i32 {
11     println!("Calling the add function");
12
13     a + b;
14 }
```

Blocks

- Anything within {}s is considered a block
- Creates a scope
- Blocks evaluate to their last expression
- (This is a functionalism, `progn`)

```
1 fn main() {
2     let y = {
3         let x = 3;
4         x + 1
5     };
6
7     println!("The value of y is: {y}");
8 }
```

Implicit Returns

- Because of this, idiomatic functions will often omit the `return keyword`
- Only works if you *OMIT SEMICOLON*
- Called a "tail expression"
- Check the compiler output if we don't

```
1 fn add(a: i32, b: i32) -> i32 {
2     println!("Calling the add function");
3
4     a + b; // fails
5     a + b // works
6 }
```

Paradigms

- Rust is multi-paradigms
- Mostly procedural
- Borrows from OOP and FP
- Only when it needs to
- We will explore this later

A Caution:

- Crash course on syntax
- Passing data around does *not* function the way you think yet
- We need to talk about memory models before we can dive into how Rust handles this
- Unique approach to memory
- If you don't take the time learn it, it will hurt you

Summary

- Rust is a strong, static, multi-paradigm language
- Uses the Cargo build system
- Variables with `let`, immutable by default
- Functions with `fn` and `->`
- Blocks implicitly return their tail

Questions?